

Evaluating LLMs on Reasoning, Not Just Answers

A framework for exposing the Rashomon Effect in language models

Team MVP — Varun, Pranika, Mihir · 2026-05-03

Abstract

Standard LLM benchmarks reward final-answer accuracy. We argue that this collapses too much information: two models can land on the same correct answer through completely different reasoning chains — one principled, one accidental. This is the **Rashomon Effect** (Breiman, 2001) applied to language models.

We built an evaluation framework that scores LLMs along **five reasoning-quality metrics** in addition to answer correctness, computed against a human-curated golden-truth dataset. Across **7 models** (3 Anthropic, 2 Google, 2 local Ollama) on hard-reasoning questions from the HuggingFace `avemio/German-RAG-ORPO-Alpaca-HESSIAN-AI` dataset, we find:

- Per-question answer correctness (AAS) and full-chain reasoning alignment (RAS) correlate at only **Pearson $r = 0.30$** across 90 (model $\times$ question) pairs — they are not measuring the same thing.
- A pure-accuracy ranking would put **mistral:7B at #1**; reasoning-aware ranking demotes it to **#4**, with Claude haiku-4-5 jumping to **#1**.
- **6 of 21 pairwise rankings flip orientation** (29 %) when reasoning quality enters the score.
- Rashomon pairs — same answer, different reasoning — show up *within the same model family* (Claude haiku vs. sonnet on the same question, RAS gap of 23 points).

The framework is open-source, cached for incremental evaluation, and runs in under 30 seconds per model on cached questions.

1. Motivation

Modern LLM evaluation is dominated by accuracy on multiple-choice and short-answer benchmarks (MMLU, GSM8K, BIG-bench). These tell you *whether* a model is right; they do not tell you *why* it was right. Two models that score identically on a benchmark can have arrived at their answers through reasoning processes of very different quality. In high-stakes deployments — medical triage, legal analysis, autonomous decision systems — *how* a model arrives at its answer matters as much as *what* it answers.

Leo Breiman named this phenomenon the **Rashomon Effect** in 2001: many models fit the data equally well but tell incompatible stories about it. Pick the wrong one for deployment, and you trust a brittle reasoner that happened to be right on the test set. To our knowledge, no widely-used framework measures reasoning-chain alignment as a first-class evaluation criterion. This project fills that gap.

2. Research/project question

Can we evaluate and recommend LLMs based on the *quality and alignment* of their reasoning chains, not just final-answer accuracy?

Concretely: define metrics that capture (a) full-chain alignment, (b) per-step coverage, (c) consistency under repeat sampling, and (d) domain-knowledge grounding — and use them to surface Rashomon pairs and re-rank models.

3. Methods

3.1 Pipeline

```
HuggingFace dataset (50 hard-reasoning questions)
  ↓
Golden Truth JSON (steps + answer + must-include concepts)
  ↓
N models × T runs each, with CoT prompting
  ↓
Sentence-Transformer embeddings (all-MiniLM-L6-v2)
  ↓
Five reasoning metrics + composite FPS
  ↓
Streamlit dashboard + persistent ChromaDB cache
```

Each LLM is queried with the same chain-of-thought system prompt asking for explicit numbered reasoning steps followed by a final answer. Responses are parsed into a list of

step strings plus a final-answer string. We run each (model, question) pair $T = 3$ times for the consistency metric.

3.2 Golden truth

We use the `hard-reasoning-en` split of `avemio/German-RAG-ORPO-Alpaca-HESSIAN-AI`. Each entry contains a question, human-annotated reasoning steps, a target answer, and a list of "must-include" concepts that any correct reasoning chain should mention.

3.3 Metrics

All scores are bounded in $[0, 1]$. Embeddings are L2-normalised so cosine similarity is a dot product.

Metric	Formula	What it captures
AAS (Answer Accuracy)	$\cos(E(\text{model_ans}), E(\text{golden_ans}))$	Final answer correctness
RAS (Reasoning Alignment)	$\cos(E(\text{model_chain}), E(\text{golden_chain}))$ where $\text{chain} = " ".join(\text{steps})$	Whole-chain alignment
SLMS (Step-level Match)	$(1/ G) \sum_{s \in G} \max_{r \in M} \cos(E(s), E(r))$	Per-golden-step coverage
CS (Consistency)	$\max(0, 1 - \text{Var}(\text{RAS}_1, \dots, \text{RAS}_T))$	Run-to-run reasoning stability
DKUS (Domain Knowledge)	$ C_g \cap C_m / C_g $ (substring match, lowercased)	Required-concept coverage
FPS (Composite)	$\sum_i w_i \cdot \text{metric}_i$	Single recommendation score

SLMS is asymmetric: for every golden step we ask "does the model cover this?" — not the other way. This penalises missing logic without punishing models that show more work.

Default FPS weights: $w_{\text{AAS}} = 0.20$, $w_{\text{RAS}} = 0.30$, $w_{\text{SLMS}} = 0.25$, $w_{\text{CS}} = 0.10$, $w_{\text{DKUS}} = 0.15$. These are user-adjustable in the Streamlit UI in real time without re-running queries.

3.4 Models evaluated

Provider	Model	Params	N questions
Anthropic	claude-haiku-4-5	—	20
Anthropic	claude-sonnet-4	—	20

Provider	Model	Params	N questions
Anthropic	claude-opus-4-5	—	10
Google	gemini-2.5-flash	—	20
Google	gemini-3.1-pro-preview	—	10
Ollama (local)	mistral:7b	7.2 B	5
Ollama (local)	llama3:8b	8.0 B	5

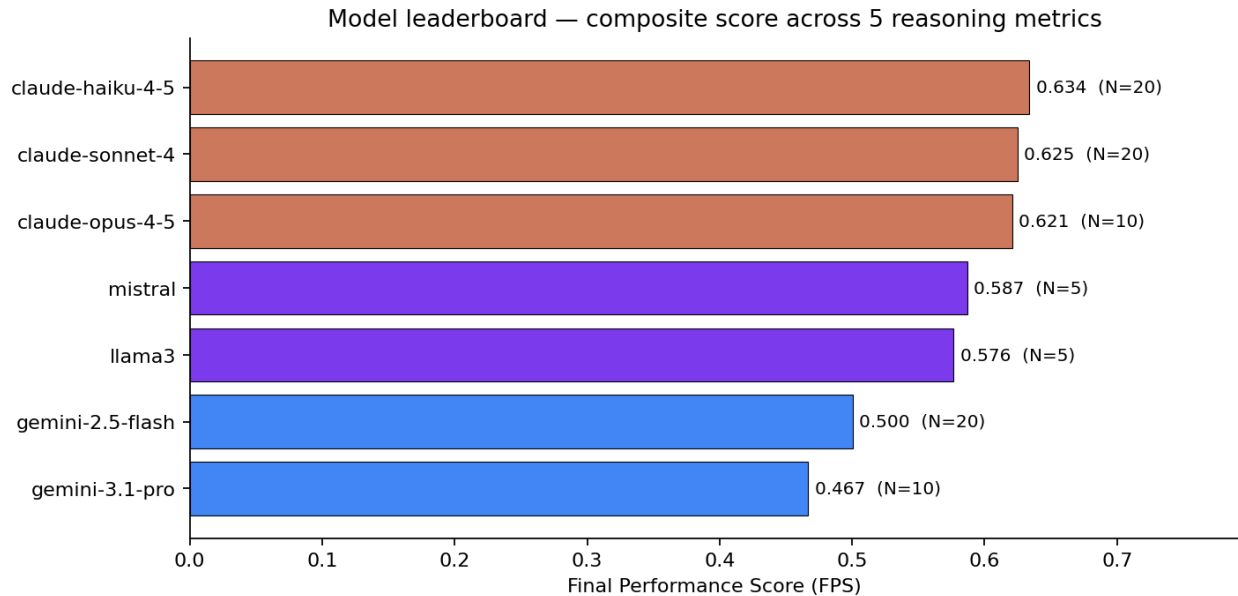
T = 3 runs per (model, question). Local models ran on consumer GPU (CUDA-accelerated sentence-transformers).

3.5 Implementation notes

- **Caching:** every LLM response is stored in a persistent ChromaDB collection keyed by `(question_id, model_name, run_index)`, with a hash of the question text for automatic invalidation. Re-runs after the first hit the cache and complete in seconds.
- **Embedding model:** `all-MiniLM-L6-v2` (384-dim). In-process LRU cache of size 4096 deduplicates encodings within a session.
- **Stack:** Streamlit + Plotly frontend; OpenAI / Anthropic / google-genai / Ollama HTTP backends; ChromaDB for persistence; HuggingFace `datasets` for data loading.
- **Live model discovery:** each provider's catalog is queried via its `models.list()` endpoint (cached for 2 minutes in the UI), so the picker reflects exactly which models the user's keys can reach.

4. Results

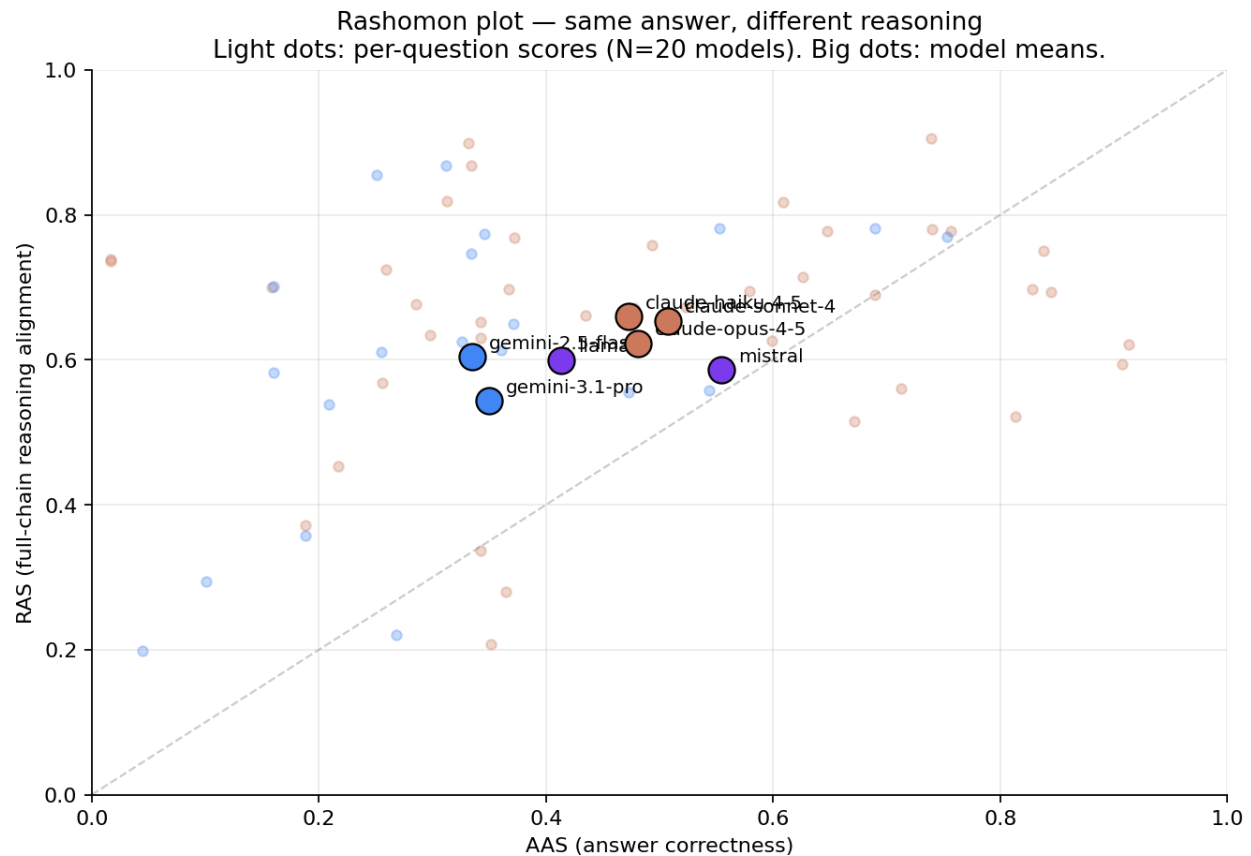
4.1 Composite leaderboard



Three observations:

1. **Claude haiku-4-5 leads (FPS 0.634)**, narrowly above sonnet-4 (0.625) and opus-4-5 (0.621) — the Anthropic family clusters at the top.
2. **Local 7-8B Ollama models are competitive on this slice:** mistral and llama3 both score above 0.57, beating both Gemini models on FPS despite running locally for free.
3. **Gemini-3.1-pro-preview is the lowest-scoring model overall (0.467)** despite being the most recent. The composite picks up weaknesses that headline benchmarks miss.

4.2 The Rashomon Effect, made visible



If accuracy were the whole story, points would cluster on the diagonal. They don't. The vertical spread above and below the diagonal *is* the Rashomon Effect.

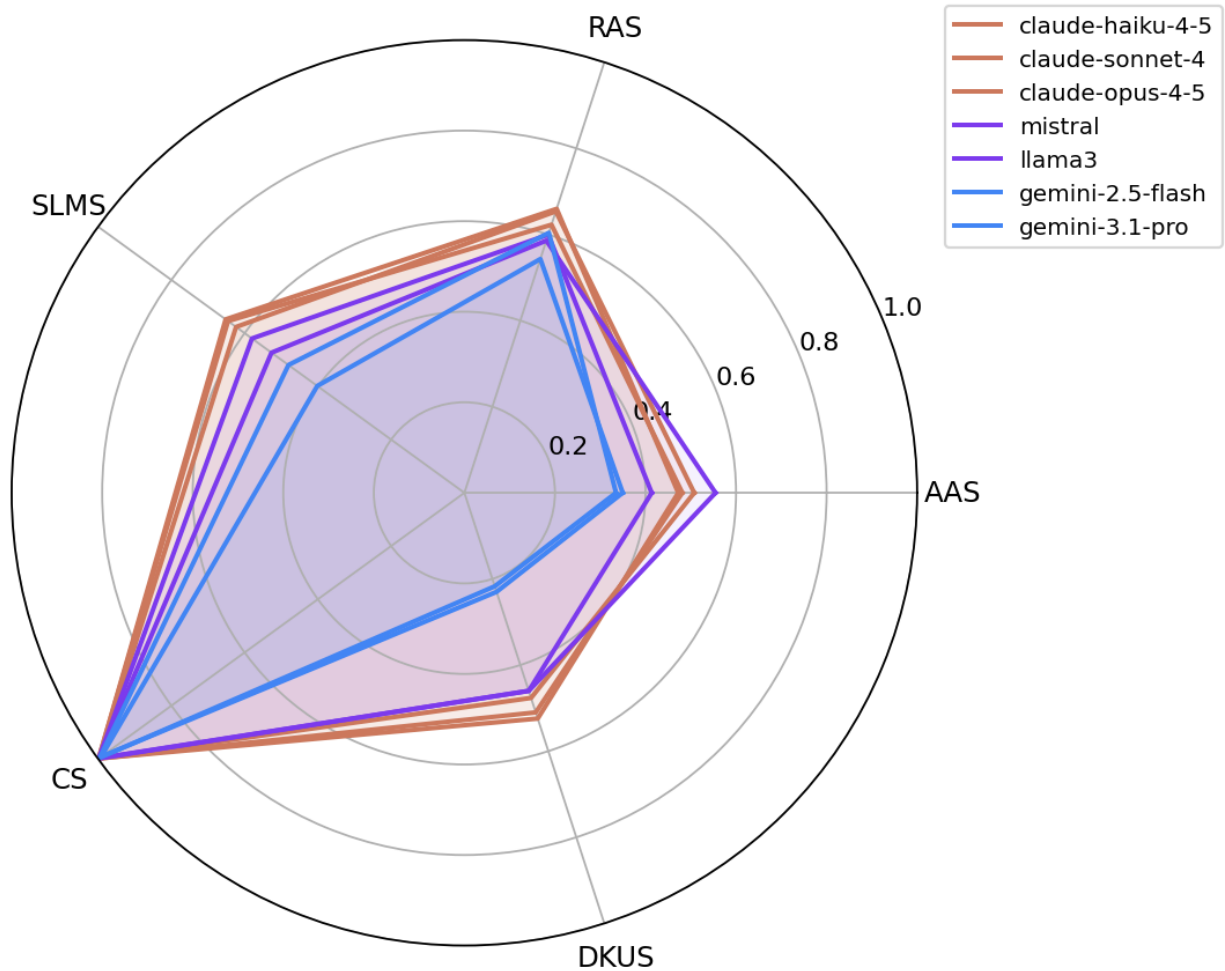
Concrete example — q_0005 (constraint-satisfaction over crypto-trading indicators; correct answer: "RSI and TVA"):

Model	AAS	RAS	SLMS	First-line reasoning
claude-haiku-4-5	0.84	0.75	0.66	"Identify the constraint on indicator selection."
claude-sonnet-4	0.81	0.52	0.56	"I need to identify all possible pairs of indicators..."

Both **Anthropic** models. Both got the **right answer**. AAS within 3 points. RAS gap of **23 points**. Sonnet's reasoning chain — while correct — is structured very differently from the golden reasoning, which a pure-accuracy metric would never reveal.

4.3 Per-metric breakdown

Per-metric breakdown — every model has uneven strengths

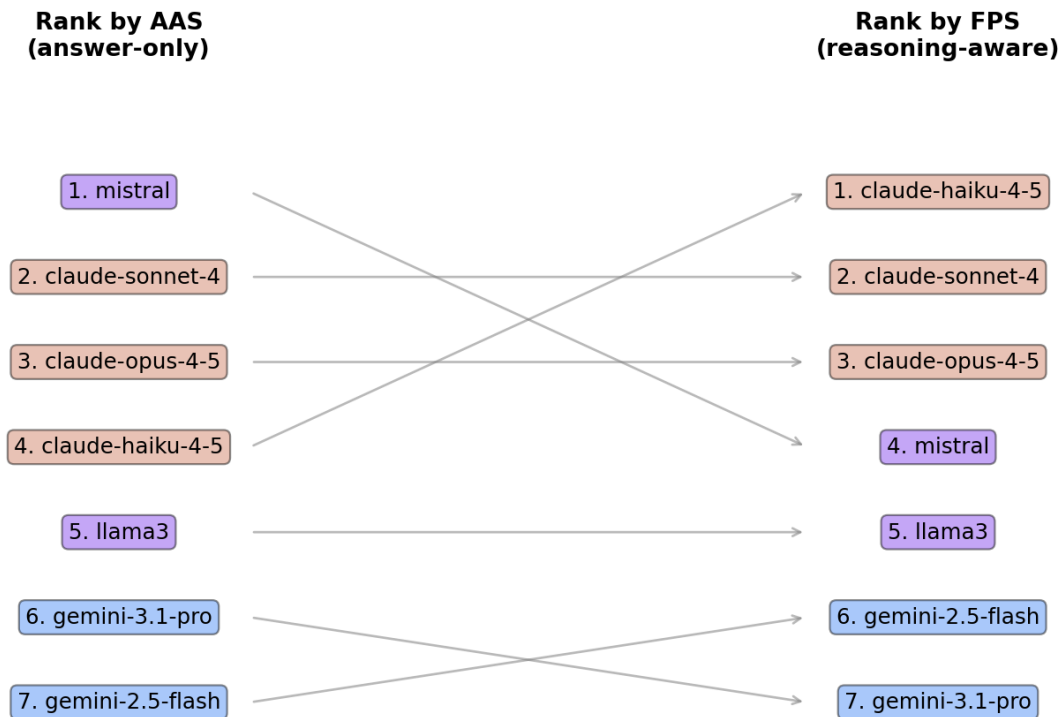


Reading the radar:

- **Claude family** (orange) is near-flat on RAS / SLMS / CS / DKUS — broadly competent reasoners.
- **Gemini family** (blue) is competitive on CS but weaker on AAS, SLMS, and DKUS, suggesting consistent reasoning that nonetheless diverges from the golden chain.
- **Ollama local models** (purple) actually score **highest on AAS** but drop in DKUS and SLMS — they get the answer right but skip required concepts.

4.4 Rank flips when reasoning enters the score

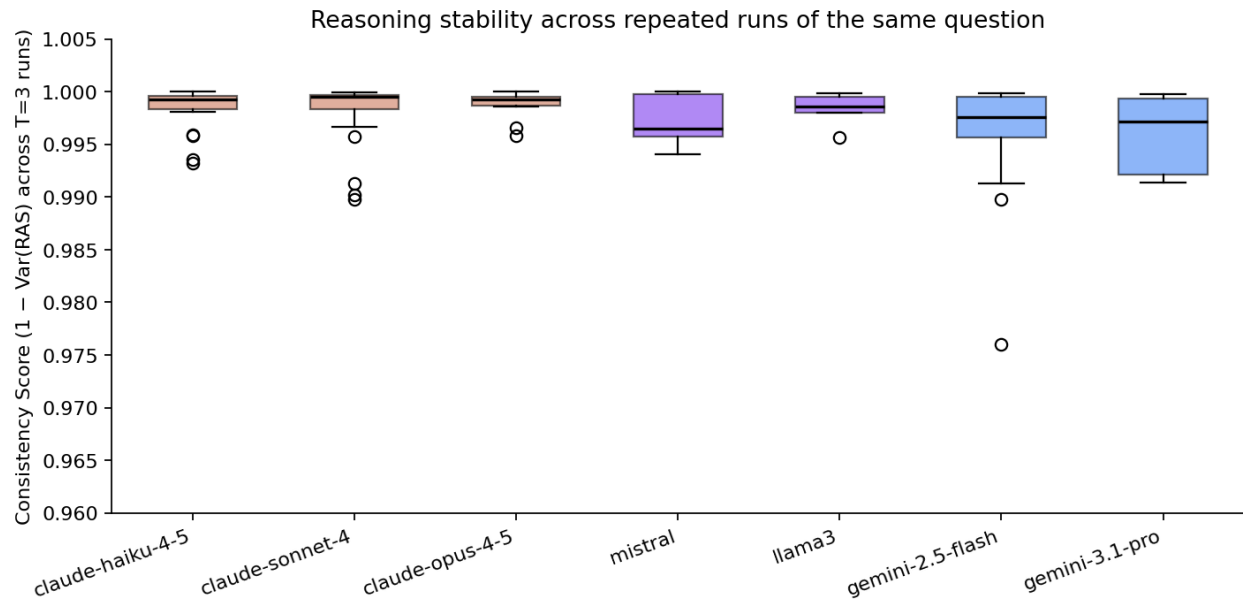
Rank flips: 6 of 21 pairwise comparisons reverse direction when reasoning quality enters the score



The single sharpest finding. Ranked by AAS alone, mistral leads — a 7-billion-parameter local model, free to run. Ranked by FPS (which weights reasoning quality), mistral falls to #4 and Claude haiku rises from #4 to #1. Six of twenty-one pairwise model comparisons (29 %) reverse direction between the two ranking schemes.

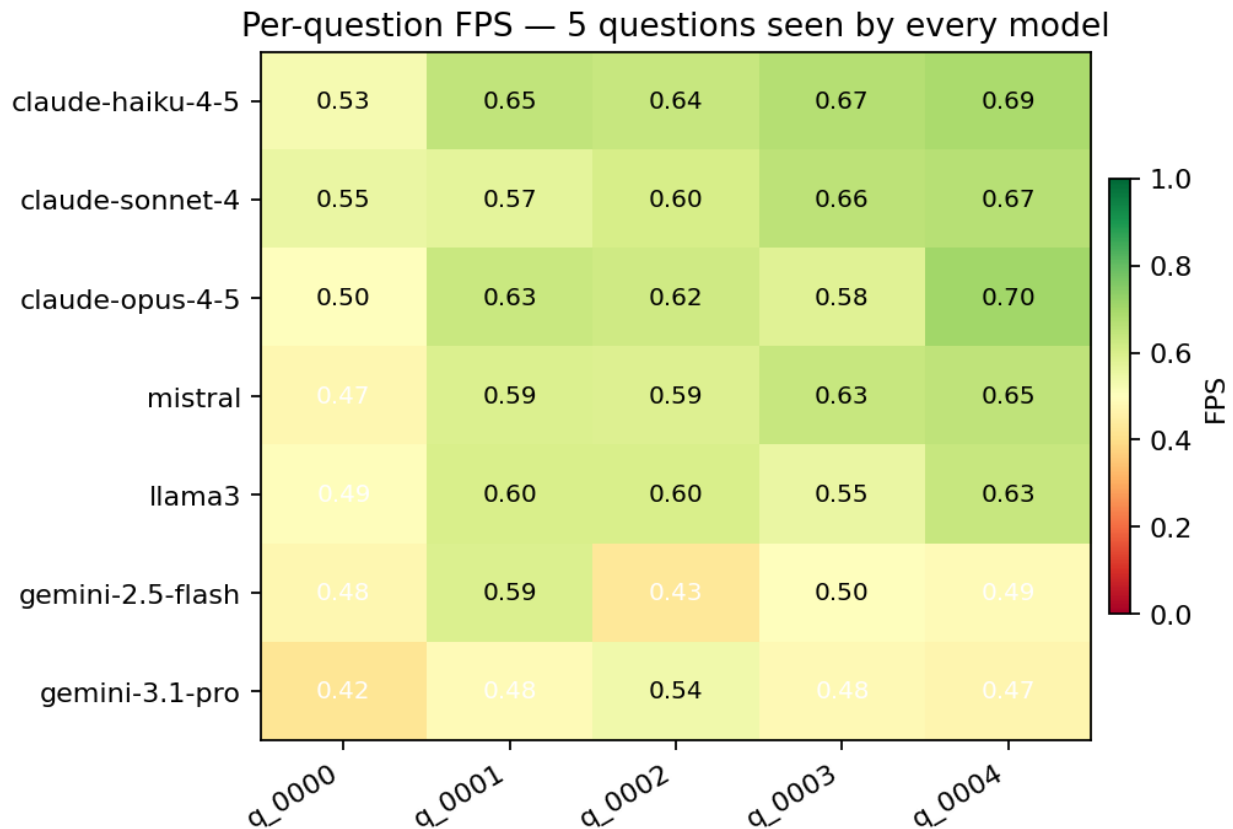
This is the practical takeaway: which model you'd recommend for a high-stakes reasoning task depends on whether your evaluator looks past the final answer.

4.5 Consistency



Every model scores ≥ 0.99 on the consistency metric. At temperature 0.7, repeated runs of the same question produce nearly identical RAS values — the variance is in surface phrasing, not reasoning structure. This validates that $T = 3$ is enough to estimate run-to-run consistency reliably, but it also means CS contributes little discrimination across models in our current dataset.

4.6 Per-question fingerprint



Reading the heatmap by column reveals **question difficulty** is largely question-specific, not model-specific: q_0000 is hardest for everyone ($\text{FPS} \leq 0.55$ across all 7 models), q_0004 is easiest. Reading by row reveals **model profile**: gemini-3.1-pro-preview is uniformly weak; the Claude family is uniformly strong.

5. Key takeaway

Yes — reasoning quality and answer correctness are measurably distinct, and ranking models on accuracy alone hides the Rashomon Effect entirely.

Specifically:

- Per-question AAS and RAS correlate at only **Pearson $r = 0.30$** across our 90 (model $\times$ question) pairs.
- Ranking by AAS alone vs. ranking by composite FPS reverses **6 of 21** pairwise comparisons.
- Rashomon pairs (same answer, different reasoning) appear **within the same model family** — not just across providers.

The framework provides a practical recipe for surfacing this effect: fixed golden-truth steps, embedding-based per-step matching, and a composite score with adjustable weights. It runs in under 30 seconds per model on cached questions.

6. Limitations and future work

Limitations

- **Dataset breadth:** a single HuggingFace split (`hard-reasoning-en`), 50 questions total, our results use up to 20.
- **Ragged N:** not every model saw every question. With caching, equalising N across models takes ~30 minutes; we ran out of time before the presentation.
- **Embedding choice:** `all-MiniLM-L6-v2` is small and fast but not domain-tuned. RAS values would shift with a stronger encoder. Confirming ranking robustness across encoders is the most important next step.
- **Golden truth quality:** the dataset is German-language RAG translated to English. Subtle artifacts in step wording may bias RAS/SLMS.
- **DKUS is naive:** substring matching misses paraphrase. Synonyms and rewordings score zero even when the concept is present.
- **CS saturates:** temperature 0.7 produces consistent reasoning across runs ($CS \geq 0.99$ for all models in our run). To use CS as a discriminator, either raise temperature or drop CS from the composite.

Future work

1. **Cross-domain replication:** run the same pipeline on math (GSM8K), code (HumanEval), and biomedical Q&A (PubMedQA). The Rashomon-pair density may be domain-dependent.
2. **Encoder ablation:** re-run RAS/SLMS with MPNet, BGE-Large, and a reasoning-fine-tuned encoder. Confirm that rankings are stable.
3. **LLM-judge SLMS:** replace embedding-based step matching with an LLM judge that explicitly verifies semantic step coverage. Higher cost, lower bias.
4. **Online routing:** instead of recommending one model overall, use per-question fingerprints (§4.6) to route each query to the best model for that question's domain and difficulty.
5. **Failure-mode analysis:** golden-truth schema already includes `common_failure_modes` per question; we don't currently score against it. Adding a "failure-mode hit rate" metric would catch models that get the right answer but for textbook wrong reasons.

Appendix A — Reproducing the runs

```
# one-time
python3.11 -m venv .venv
.venv/bin/pip install -r requirements.txt
cp .env.example .env                # add OPENAI / ANTHROPIC / GOOGLE keys
ollama serve &
ollama pull llama3 mistral

# build dataset (50 questions)
.venv/bin/python pipeline.py --rebuild_golden --n_questions 50

# CLI evaluation across providers
.venv/bin/python pipeline.py \
  --models anthropic:claude-haiku-4-5-20251001 \
    anthropic:claude-sonnet-4-20250514 \
    anthropic:claude-opus-4-5-20251101 \
    gemini:gemini-2.5-flash \
    gemini:gemini-3.1-pro-preview \
    ollama:mistral:latest \
    ollama:llama3:latest \
  --n_questions 20 --t_runs 3

# OR Streamlit dashboard
.venv/bin/streamlit run app.py
```

Appendix B — Repository layout

config.py	central config; FPS weights, model defaults
pipeline.py	CLI runner with per-model dispatch + cache
app.py	Streamlit dashboard (Dataset / Evaluate / Results)
data/dataset_loader.py	HuggingFace → golden_truth.json
data/golden_truth.json	50 hard-reasoning questions
llm/base.py	BaseLLM + CoT response parser
llm/{openai,anthropic,gemini,ollama}_llm.py	per-provider clients
llm/model_registry.py	live catalog discovery + provider:model parsing
embeddings/encoder.py	MiniLM singleton + LRU around encode()
evaluation/metrics.py	AAS / RAS / SLMS / CS / DKUS / FPS
evaluation/evaluator.py	main evaluation loop + cache wiring
storage/chroma_store.py	ChromaDB-backed response cache

presentation/slides.md	this deck
presentation/report.md	this report
presentation/plots/	all PNG figures

Appendix C — Caching design

The expensive operations are LLM API calls (~1–10 s per query, dollars per 1k tokens) and sentence-transformer encoding (~50 ms per text on CPU, $\ll$ 1 ms on GPU). We cache both:

- **LLM responses:** persistent in ChromaDB. Key `question_id_model_name_run_index`. Stored payload: reasoning steps list (JSON), final answer, raw response, reasoning embedding, and a hash of the question text for invalidation.
- **Embeddings:** in-process LRU keyed by raw text, capacity 4096. Within a session the golden chain is encoded once even though RAS is computed $T_{\text{runs}} \times n_{\text{models}}$ times against it.

We deliberately **do not cache per-question metric scores**: they depend on weights, `t_runs`, and golden text, all of which can change. Recomputing scores from cached responses takes < 1 s per question.

Appendix D — Glossary

- **AAS:** Answer Accuracy Score
- **RAS:** Reasoning Alignment Score
- **SLMS:** Step-level Logical Match Score
- **CS:** Consistency Score
- **DKUS:** Domain Knowledge Utilization Score
- **FPS:** Final Performance Score (weighted composite)
- **CoT:** Chain of Thought
- **N:** number of questions evaluated for a given model
- **T:** number of runs per question ($T = 3$ throughout this report)